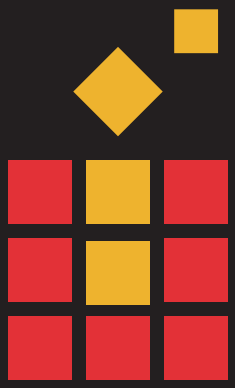
data_rate



Application Programming Interface Guide

for
WAVE Device

Revision: 1.1

Table of Contents

1. Objective 1

2. Scope 2

3. Acronyms 3

4. Command-line interface (CLI) Commands 5

4.1. GPS Command 5

4.2. IEEE 1609.3 Test 6

4.2.1. IEEE 1609.3 Test Initialization 6

4.2.2. IEEE 1609.3 Test Exit 6

4.3. Channel Service 7

4.3.1. Channel Service Create 7

4.3.2. Channel Service Delete 8

4.4. Provider Service 9

4.4.1. Provider Service Create 9

4.4.2. Provider Service Delete 10

4.5. User Service 11

4.5.1. User Service Create 11

4.5.2. User Service Delete 12

4.6. WSM Service 13

4.6.1. WSM Service Create 13

4.6.2. WSM Service Delete 13

4.6.3. WSM Send 14

4.7. MIB Status 16

4.8. J2735 Message 17

4.8.1. BSM Receive 17
4.8.2. BSM Transmit 17

4.9. Security Test (1609.2) 19

4.10. Firmware Upgrade 20

5. IEEE 1609.3 Application Programming Interfaces (APIs) 21

5.1. WAVE Management Entity (WME) Initialization APIs 22

5.1.1. WME Initialization 22
5.1.2. WME De-initizliation 23

5.2. Provider Service APIs 24

5.2.1. Provider Service Creation 24
5.2.2. Provider Service Changing 27
5.2.3. Provider Service Deletion 28

5.3. User Service APIs 29

5.3.1. User Service Creation 29
5.3.2. User Service Deletion 31

5.4. Channel Service APIs 33

5.4.1. Channel Service Creation 33
5.4.2. Channel Service Deletion 34

5.5. WSM Service APIs 36

5.5.1. WSM Service Creation 36
5.5.2. WSM Service Deletion 37

5.6. Data Plane APIs 38

5.6.1. WSM Sending 38
5.6.2. WSM Receiving 40

5.7. Event Receive API 43

5.8. MIB Information Get API 45

5.9. Example Code 47

6. IEEE 1609.2 Application Programming Interfaces (APIs) 48

6.1. Security Credential Management 49

6.1.1. Cryptomaterial Management 49

6.1.2. Symmetric Cryptomaterial Management 54

6.1.3. CA Certificate 56

6.1.4. Certificate Generation 56

6.2. Security Message Service 58

6.2.1. Signed Data Generation 58

6.2.2. Data Preprocessing 61

6.2.3. Verification 63

6.2.4. Encryption 67

6.2.5. Decryption 69

6.2.6. Encoding 71

6.2.7. Decoding 73

6.3. Example Code 76

7. SAE J2735 Application Programming Interfaces (APIs) 77

7.1. Basic Safety Message (BSM) 77

7.1.1. BSM Encoding 78

7.1.2. BSM Decoding 79

7.1.3. Example Code 79

1. Objective

Unex WAVE device contains V2X elements of DSRC/GNSS/HSM with pre-integrated IEEE 802.11p driver and IEEE 1609.x protocol stack (version 2016) running on ThreadX RTOS.

This document introduces command-line interface (CLI) commands and application programming interfaces (APIs) provided by Unex WAVE SDK. The CLI commands help customers to configure the WAVE device and do quick test (see “Quick Start Guide for WAVE Device” for more detail). The APIs help customer to develop their own application over IEEE 1609.x protocol stack.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

2. Scope

Supported product:

» OBU-201U

» RSU-101U

Supported protocol stack:

» Unex WAVE-SDK-v1007 or above (apply Autotalk SDK-4.11.1)

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

3. Acronyms

OBU	Onboard Unit
RSU	Roadside Unit
DSRC	Dedicated Short Range Communication
CCH	Control Channel
SCH	Service Channel
PSC	Provider Service Context
PSID	Provider Service Identifier
WAVE	Wireless Access in Vehicular Environments
WSA	WAVE Service Advertisements
WSMP	WAVE Short Message Protocol
WME	WAVE Management Entity
BSM	Basic Safety Message
SPAT	Signal Phase and Timing Message
MAP	Map Data
TIM	Traveler Information Message
RSA	Road Side Alert
RTCM	Radio Technical Commission for Maritime Services
CSR	Common Safety Request Message
NMEA	National Marine Electronics Association
ICA	Intersection Collision Alert Message
EVA	Emergency Vehicle Alert Message
PDM	Probe Data Message

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

SSM	Signal Status Message
SRM	Signal Request Message
PVD	Probe Vehicle Data Message
PSM	Pedestrian Safety Message

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4. Command-line interface (CLI) Commands

4.1. GPS Command

The GPS command.

Syntax:

```
gps <command>
```

Parameter	Valid Range	Description
command	“show”	Currently only one valid command “show”, to display GPS status

Example:

```
ate> gps show
GPSINFO: time: Mon May 28 06:31:54 2018

    latitude 24.8085697, longitude 121.0360252
    altitude 130.8300000 m
    ground speed 0.0514444 m/s
    angle 0.0000000
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.2. IEEE 1609.3 Test

4.2.1. IEEE 1609.3 Test Initialization

To initialize the 1609.3 test environment

Syntax:

```
dot3_test_init
```

Example:

```
ate> dot3_test_init
wme_init success, handle = 1
```

After successful execution, a valid handle id will return. It will be applied by the following service creation/deletion.

4.2.2. IEEE 1609.3 Test Exit

To exit the 1609.3 test.

Syntax:

```
dot3_test_exit
```

Example:

```
ate> dot3_test_exit
ret = 26
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.3. Channel Service

4.3.1. Channel Service Create

To create a channel service. Should be called after getting a valid handle id from "dot3_test_init"

Syntax:

```
dot3_channel_serv_create <handle> <radio_number> <channel_number> <slot> <priority>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from dot3_test_init
radio_number	1 - 2	The radio number
channel_number	172 174 176 178 180 182 184	The channel applied for the service
slot	1: CCH 2: SCH 3: BOTH	Channel access option used by service. CCH only, SCH only, or alternating access
priority	0 ~ 63	Priority of user service. Lower value has higher priority

Example:

```
ate> dot3_channel_serv_create 1 1 172 3 5
index = 0, channel 178 BOTH delete
dev_id = 0, channel = 170
index = 0, channel 172 BOTH add
dev_id = 0, channel = 172
Channel 172 assigned ...
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
wme_channel_serv_create success, serv_index = 0
```

After successful execution, a valid service index will return.

4.3.2. Channel Service Delete

To delete a channel service

Syntax:

```
dot3_channel_serv_delete <handle> <serv_index>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
serv_index	0 ~ 255	Service index return from `dot3_channel_serv_create`

Example:

```
dot3_channel_serv_delete 1 0
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.4. Provider Service

4.4.1. Provider Service Create

To create a provider service to send WSA that indicate the dedicate channel and radio number to send/receive WSM. Should be called after getting a valid handle id from "`dot3_test_init`"

Syntax:

dot3_provider_serv_create <handle> <psid> <repeat_rate> <radio_number> <service_channel> <channel_access> <priority> <is_secure> <sign_lifetime>

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
psid	1 ~ 127	PSID of desired service
repeat rate	0 ~ 255	Repeat rate of WSA packets
radio_number	1 - 2	Radio number of the service
service_ channel	172 174 176 180 182 184	Channel number of desired service
channel_ access	0: BOTH 1: SCH 2: CCH	Channel access option used by service. CCH only, SCH only, or alternating access
priority	0 ~ 63	Priority of user service. Lower value has higher priority
is_secure	0: no secured WSA 1: secured WSA	Security enabling status

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

sign_lifetime	10 ~ 30000 ms	Life time of signature

Example:

```
ate> dot3_provider_serv_create 1 123 1 2 172 0 5 0 10
index = 1, channel 172 BOTH add
dev_id = 3, channel = 172
Channel 172 assigned ...
wme_provider_serv_create success, serv_index = 0
```

After successful execution, a valid service index will return.

4.4.2. Provider Service Delete

To delete a provider service.

Syntax:

```
dot3_provider_serv_ delete <handle> <serv_index>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
serv_index	0 ~ 255	Service index return from `dot3_provider_serv_create`

Example:

```
dot3_provider_serv_delete 1 0
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.5. User Service

4.5.1. User Service Create

To create a user service to receive WSA (if PSID & channel number match). Should be called after getting a valid handle id from “dot3_test_init”

Syntax:

```
dot3_user_serv_create <handle> <psid> <request_type>
<priority> <radio_number> <channel_number>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from dot3_test_init
psid	1 ~ 127	PSID of desired service
request_type	0: Auto-access on service match 1: NO SCH Access	Indicates the requested scheduler behavior. Provide SCH access request while match the request, or not provide SCH access automatically
priority	0 ~ 63	Priority of user service. Lower value has higher priority
radio_number	1 - 2	Radio number of the service
channel_number	172 174 176 180 182 184	Channel number of desired service

Example:

```
ate> dot3_user_serv_create 1 123 0 5 2 172
wme_user_serv_create success, serv_index = 0
```

After successful execution, a valid service index will return.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.5.2. User Service Delete

To delete a user service.

Syntax:

```
dot3_user_serv_delete <handle> <serv_index>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
serv_index	0 ~ 255	Service index return from `dot3_user_serv_delete`

Example:

```
dot3_user_serv_delete 1 0
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.6. WSM Service

4.6.1. WSM Service Create

To create a WSM service to receive WSM. Should be called after getting a valid handle id from "`dot3_test_init`"

Syntax:

```
dot3_wsm_serv_create <handle> <psid>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
psid	1 ~ 127	PSID of desired service

Example:

```
ate> dot3_wsm_serv_create 1 123
wme_wsm_serv_create success, serv_index = 0
```

After successful execution, a valid service index will return.

4.6.2. WSM Service Delete

To delete a user service.

Syntax:

```
dot3_wsm_serv_delete <handle> <serv_index>
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
serv_index	0 ~ 255	Service index return from `dot3_wsm_serv_create`

Example:

```
dot3_wsm_serv_delete 1 0
```

4.6.3. WSM Send

To sends WSM packets with dedicated PSID on dedicate channel and radio number. Should be called after getting a valid handle id from "`dot3_test_init`". The data packet sent is dummy data and it's for testing purpose only.

Syntax:

```
dot3_wsm_send <handle> <psid> <radio_number> <channel_number> <data_rate> <tx_power> <dest_mac> <size> <interval> <num>
```

Parameter Table:

Parameter	Valid Range	Description
handle	0 ~ 255	Handler id return from `dot3_test_init`
psid	1 ~ 127	PSID of desired service
radio_number	1 - 2	Radio number of the service
channel_number	172 174 176 180 182 184	Channel number of sending WSMv2 packets
data_rate	6, 9, 12, 18, 24, 36, 48, 54	Data rate of sending WSMv2 packets

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

tx_power	12 ~ 25	Tx power of sending WSMv2 packets (unit: dBm)
dest_mac	format: XX:XX:XX:XX:XX:XX	Peer mac address of sending WSMV2 packets
size	0 - 1399	Size of sending packets (unit: byte)
interval	0 - 1000	Interval of sending packets (unit: ms)
num	1 - 10000	Number of sending packets

Example:

```
dot3_wsm_send 1 123 1 172 6 18 FF:FF:FF:FF:FF:FF 1000 100
10
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.7. MIB Status

To retrieve the dot3 MIB information form WAVE device. This command can be issued at any time to retrieve current MIB information of target device.

Syntax:

```
dot3_show_mib
```

Example:

```
ate> dot3_show_mib
wme_init success, handle = 2
*** Provider entry ***
*** User entry ***
*** Channel entry ***
index = 0
priority  = 5
channel  = 172
slot  = 3

*** Wsm entry ***
*** Available entry ***
```

As shown above, we can see a ***Channel entry*** in MIB.

Note:

On receiving WSA, relevant information will be displayed in ***Available entry***

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.8. J2735 Message

To transmit/receive J2735 BSM message. All other J2735 message (SPAT/MAP/TIM/RSA/RTCM/CSR/NMEA/ICA/EVA/PDM/SSM/SRM/PVD/PSM) can also apply the same command style to replace the acronyms

4.8.1. BSM Receive

To receive BSM packets

Syntax:

```
rxBsm <psid>
```

Parameter Table:

Parameter	Valid Range	Description
psid	1 ~ 127	PSID of desired BSM packets

Example:

```
rxBsm 123
```

4.8.2. BSM Transmit

To transmit BSM packets

Syntax:

```
txBsm <psid> <data_rate> <tx_power> <dest_mac> <interval>
<num>
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Parameter Table:

Parameter	Valid Range	Description
psid	1 ~ 127	psid of sending BSM packets
data_rate	6 9 12 18 24 36 48 54	Data rate of sending BSM packets
tx_power	12- 25, unit:dBm	tx power level of sending BSM packets
dest_mac	format: XX:XX:XX:XX:XX:XX	Peer mac address of sending BSM packets
interval	0 - 1000, unit:ms	The interval of sending packets
num	1 - 10000	The number of sending packets

Example:

```
txBsm 123 6 18 FF:FF:FF:FF:FF:FF 1000 1
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.9. Security Test (1609.2)

To do a 1609.2 encode/decode testing.

Syntax:

```
test_us_sec <psid> <sign_flag> <encrypt_flag> <verify_flag>
<msg_len> <repeat>
```

Parameter Table:

Parameter	Valid Range	Description
psid	1 ~ 127	PSID to be encoded if <sign_flag> is set
sign_flag	0: not sign 1: sign	Indicates whether to sign the message or not
encrypt_flag	0: not encrypt 1: encrypt	Indicates whether to encrypt the message or not
verify_flag	0: only decode, not verify 1: decode and verify	Indicates whether to verify the message or not if it is a signed message
msg_len	1 ~ 200	Length of testing message
repeat		Number of testing rounds

Example:

```
test_us_sec 125 1 1 1 128 1
```

After issue, it will load the testing root certificate/key, and PCA certificated/key, hard coded in the code, and generate an end-entity certificate. Then random generate a data size 128 bytes. Sign and encrypt the data using the end-entity certificate to generate encoded 1609.2 data. And last decrypt and verify the encoded data to get the original raw 128 bytes data.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

4.10. Firmware Upgrade

Please refer to chapter 10 of “Quick Start Guide for WAVE Device” for firmware upgrade procedure and command usage.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5. IEEE 1609.3 Application Programming Interfaces (APIs)

This chapter describe the detail of APIs provided by 1609.3 stack. The function prototype of APIs defined in "`/include/dot3_api.h`"

The process for using IEEE 1609.3 networking services API shown in Figure 1.

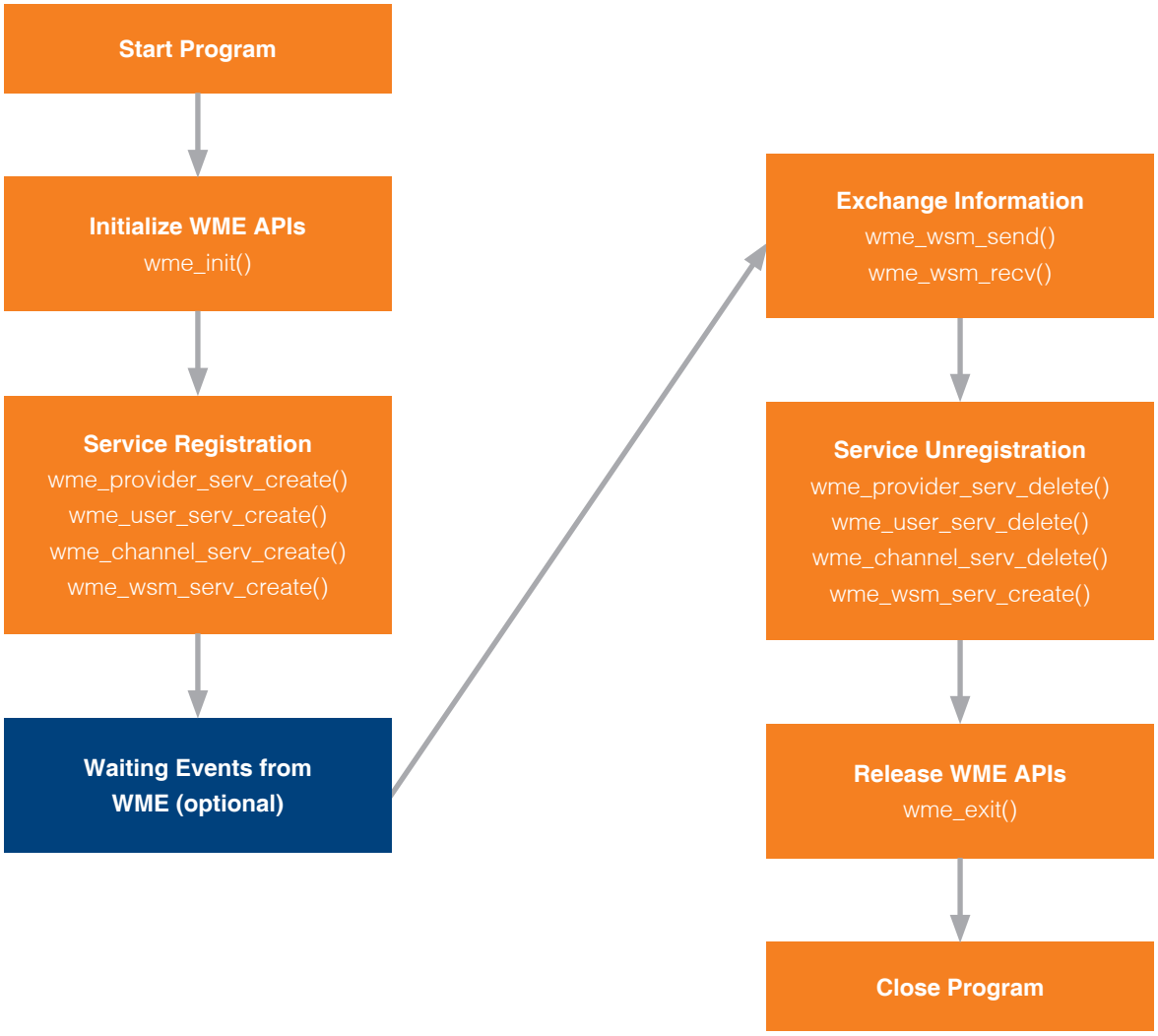


Figure 1 - 1609.3 API Flow

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.1. WAVE Management Entity (WME) Initialization APIs

Before access any services we have to initialize the WAVE management entity (WME). And after we done those services, we have to de-initialize WME also.

5.1.1. WME Initialization

This function will initialize the WME service and get at handle id for the service. The handle ID is applied in the further WME services, such as channel service, provider service, user service, and WSM service.

Prototype:

```
int wme_init(wme_handle_t *handle);
```

Name	*wme_init*		
Description	Initialize WME service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Output	wme_handle_t *	Handle ID for WME service

Type:

```
typedef uint16_t wme_handle_t;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.1.2. WME De-initizliation

This function will de-initialize the WME service indicated by the handle ID

Prototype:

```
int wme_exit(wme_handle_t *handle);
```

Name	*wme_exit*		
Description	De-initialize WME service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.2. Provider Service APIs

There are 3 APIs for provider service: creation, changing, and deletion.

5.2.1. Provider Service Creation

Before provider service creation, a valid handle ID should be obtained from "`wme_init`". After successful provider service creation, a valid service ID will be return to be used in following changing or deletion. And WME will generate a corresponding WSA and send the WSA periodically

Prototype:

```
int wme_provider_serv_create(wme_handle_t *handle, unsigned
short *serv_index, wme_provider_info_t *info);
```

Name	*wme_provider_serv_create*		
Description	Create provider service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Output	unsigned short *	Service index of provider service
info	Input	wme_provider_ info_t *	Parameters of provider service

Structure Type:

```
typedef struct provider_info{
    unsigned char       action;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
    unsigned char       dest_mac[MAC_SIZE];  /* No support */
    unsigned char       wsa_type;  /* enum WSA_TYPE */
    unsigned int        psid;
    unsigned char       psc[PSC_SIZE];
    unsigned char       psc_len;
    unsigned char       service_priority;
    unsigned char       radio_number;  /* RADIO_0 or RADIO_1 */
    unsigned char       service_channel;
     unsigned char        wsa_channel;  /* NO Support, only 178
now */
    unsigned char       channel_access;  /* enum ACCESS_SLOT */
    unsigned char       repeat_rate;
    unsigned char       ip_service;
    unsigned char       ipv6_addr[IPV6_SIZE];
    unsigned short      service_port;
    unsigned char       provider_mac[MAC_SIZE];
    unsigned char       rcpi_threshold;  /* no use now */
    unsigned char       wsa_count_threshold;
    unsigned char       wsa_count_threshold_interval;
    unsigned int        info_element_indicator;
    unsigned short      certificate_id;
    unsigned short      signature_lifetime;  /* 10 ~ 30000ms */
} wme_provider_info_t;
```

Struct Field Description:

Field	Description
action	Not used.
dest_mac	The destination MAC address of WSA. Not supported currently.
wsa_type	Valid value: WSA_SCURED, WSA_UNSCURED

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

psid	The desired Provider Service Identifier (PSID) Valid value: defined by IEEE 1609.3
psc	The description is associated to the psid, and the max length is PSC_SIZE (31 bytes).
psc_len	The real length of PSC
service_priority	Service priority: 0 ~ 63. The lower value has higher priority.
radio_number	Valid value: RADIO_0 or RADIO_1
service_ channel	Valid value: 172, 174, 176, 180, 182, 184. The control channel can't be used.
wsa_channel	NO Support, only 178 now
channel_ access	Valid value: SLOT_BOTH, SLOT_SCH, SLOT_CCH
repeats	The number of WSAs to be sent per 5 seconds. The real sending rate is equal to repeats +1. Valid value: 0~ 255
ip_service	Set to 0. The IP service is not available now.
ipv6_addr	Specify the IPv6 address for the service Not supported currently.
service_port	Specify the service port for the service Not supported currently.
provider_mac	Specify the provider's mac address. It is optional. If no use, Set all the value to 0xFFFFFFFFFFFF.
rcpi_threshold	The threshold of RCPI of received WSAs. Not supported currently.
wsa_count_ threshold	It indicates the recommended minimum number of received WSAs. It is optional. If no use, set it to 0. Valid value: 0 ~ 255.
wsa_count_ threshold_ interval	It indicates the time interval over which received WSAs are counted. The unit is 100ms. It is optional. If no use, set it to 0. Then the remote user accepting the WSA will use the default interval (one second). Valid value: 0 ~ 255

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

info_element_indicator	It indicates which optional information can be inserted into WSAs. Valid value: BITMASK_PSC, BITMASK_PROVIDER_MAC, BITMASK_RCPI_THRESHOLD, BITMASK_WSA_COUNT_THRESHOLD, BITMASK_WSA_COUNT_THRESHOLD_INTERVAL
certificate_id	Id of the certificate for signing WSAs. Only for secured WSAs
signature_lifetime	The number of milliseconds over which the WSA signature should be valid. Only for secured WSAs Valid value: 10 ~ 30000

5.2.2. Provider Service Changing

After provider service created, we can change the service by given hand ID, service index (return from "*wme_provider_serv_create*"), and the fill-in provider info structure.

Prototype:

```
int wme_provider_serv_change(wme_handle_t *handle, unsigned short serv_index, wme_provider_info_t *info);
```

Name	*wme_provider_serv_change*		
Description	Change provider service content		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Input	unsigned short *	Service index of provider service return from "*wme_provider_serv_create*"
info	Input	wme_provider_info_t *	Parameters of provider service

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.2.3. Provider Service Deletion

Delete the provider service indicated by the service index

Prototype:

```
int wme_provider_serv_delete(wme_handle_t *handle, unsigned short serv_index);
```

Name	*wme_provider_serv_delete*		
Description	Delete provider service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Input	unsigned short *	Service index of provider service return from "*wme_provider_serv_create*"

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.3. User Service APIs

There are 2 APIs for user service: creation, and deletion.

5.3.1. User Service Creation

Before user service creation, a valid handle ID should be obtained from “wme_init”. After successful user service creation, a valid service ID will be return to be used in following deletion. And WME will try to match the received WSAs. If the service matches a received WSA and the priority is higher, WME will assign a specific channel for the service.

Prototype:

```
int  wme_user_serv_create(wme_handle_t  *handle,  unsigned short *serv_index, wme_user_info_t *info);
```

Name	*wme_user_serv_create*		
Description	Create user service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from “*wme_init*”
serv_index	Output	unsigned short *	Service index of user service
info	Input	wme_user_info_t *	Parameters of user service

Structure Type:

```
typedef struct user_info {
      unsigned char          action;
      unsigned char          request_type;  /* enum REQUEST_ACCESS */
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
    unsigned int          psid;
    unsigned char         psc[PSC_SIZE];
    unsigned char         psc_len;
    unsigned char         service_priority;
    unsigned char         wsa_type;
    unsigned char         radio_number;  /* RADIO_0 or RADIO_1 */

    unsigned char         channel_number;
    unsigned char         src_mac[MAC_SIZE];
    unsigned char         advertiser_id[ADVERTISER_ID_SIZE];
    unsigned char         advertiser_id_len;
    unsigned char         link_quality;  /* No support */
    unsigned char         immediate_access;     /*   Only   support
0(Channel switch) or 255(Continuous) */
} wme_user_info_t;
```

Struct Field Description:

Field	Description	Match
action	Not used.	
request_type	Valid value: ACCESS_ON_MATCH, ACCESS_NO_SCH	
psid	The receiving Provider Service Identifier (PSID)	•
psc	The description is associated to the psid, and the max length is PSC_SIZE (31 bytes).	•
psc_len	The real length of PSC	
service_priority	Service priority: 0 ~ 63. The lower value has higher priority.	
wsa_type	Valid value: WSA_UNSECURE, WSA_SCURED, WSA_BOTH, WSA_ANY	
radio_number	Valid value: RADIO_0 or RADIO_1	

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

channel_ number	The user service will match up the provider service with the channel. Valid value: 172, 174, 176, 180, 182, 184. The control channel can't be used.	•
src_mac	The user will match up the provider service with the MAC. The value 0xFFFFFFFFFFFF in all bits indicates that any MAC is accepted.	•
advertiser_id	The description is associated to the device sending WSAs, and the max length is ADVERTISER_ID_SIZE (32 bytes).	
advertiser_id _len	The real length of the advertiser_id	
link_quality	The threshold of the quality for received WSAs. Not supported currently.	
immediate_ access	Valid value: 0 (Channel switch) or 255 (Continuous)	

5.3.2. User Service Deletion

Delete the user service indicated by the service index

Prototype:

```
int wme_user_serv_delete(wme_handle_t *handle, unsigned short serv_index);
```

Name	*wme_user_serv_create*
Description	Create user service
Return Value	0: Success Other: Fail

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Parameters	Mode	Type	Parameter description
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Input	unsigned short *	Service index of user service return from "*wme_user_serv_create*"

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.4. Channel Service APIs

There are 2 APIs for channel service: creation, and deletion.

5.4.1. Channel Service Creation

Before channel service creation, a valid handle ID should be obtained from "*wme_init*". After successful channel service creation, a valid service ID will be return to be used in following deletion.

Prototype:

```
int wme_channel_serv_create(wme_handle_t *handle, unsigned
short *serv_index, wme_channel_info_t *info);
```

Name	*wme_channel_serv_create*		
Description	Create channel service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Output	unsigned short *	Service index of channel service
info	Input	wme_channel_ info_t *	Parameters of channel service

Structure Type:

```
typedef struct channel_info
{
    unsigned char       action;
    unsigned char       service_priority;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
    unsigned char       radio_number;  /* RADIO_0 or RADIO_1 */

    unsigned char       channel_number;
    unsigned char       time_slot;  /* enum TIME_SLOT */
} wme_channel_info_t;
```

Struct Field Description:

Field	Description
action	Not used.
service_priority	Service priority: 0 ~ 63. The lower value has higher priority.
radio_number	Valid value: RADIO_0 or RADIO_1
channel_ number	The channel that users desired to reserve Valid value: 172, 174, 176, 178, 180, 182, 184.
time_slot	Valid value: TIME_SLOT_0, TIME_SLOT_1, or TIME_SLOT_BOTH

5.4.2. Channel Service Deletion

Delete the channel service indicated by the service index

Prototype:

```
int wme_channel_serv_delete(wme_handle_t *handle, unsigned short serv_index);
```

Name	wme_channel_serv_delete
Description	Create channel service
Return Value	0: Success Other: Fail

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Parameters	Mode	Type	Parameter description
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Input	unsigned short *	Service index of user service return from "*wme_channel_serv_create*"

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.5. WSM Service APIs

There are 2 APIs for channel service: creation, and deletion

5.5.1. WSM Service Creation

Before WSM service creation, a valid handle ID should be obtained from "`wme_init`". After successful WSM service creation, a valid service ID will be return to be used in following deletion. And WME will receive WSM packets with the specific PSID

Prototype:

```
int  wme_wsm_serv_create(wme_handle_t  *handle,  unsigned short *serv_index, wme_wsm_info_t *info);
```

Name	*wme_wsm_serv_create*		
Description	Create WSM service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "***wme_init***"
serv_index	Output	unsigned short *	Service index of WSM service
info	Input	wme_wsm_info_t *	Parameters of WSM service

Structure Type:

```
typedef struct wsm_info
{
    unsigned char   action;
    unsigned int         psid;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
} wme_wsm_info_t;
```

Struct Field Description:

Field	Description
action	Not used.
psid	The receiving Provider Service Identifier (PSID)

5.5.2. WSM Service Deletion

Delete the channel service indicated by the service index

Prototype:

```
int wme_wsm_serv_delete(wme_handle_t *handle, unsigned short serv_index);
```

Name	wme_wsm_serv_delete		
Description	Delete WSM service		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "*wme_init*"
serv_index	Input	unsigned short	Service index of user service return from "*wme_wsm_serv_create*"

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.6. Data Plane APIs

There are 2 data plane APIs: WSM sending, and WSM receiving

5.6.1. WSM Sending

This function is used to send WSM packets on a specific channel. Before sending, a valid handle ID should be obtained from “wme_init”.

Prototype:

```
int wme_wsm_send(wme_handle_t *handle, struct out_wsm *wsm);
```

Name	*wme_wsm_send*		
Description	Send WSM packet		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from “*wme_init*”
serv_index	Input	struct out_wsm *	WSM packet to send

Structure Type:

```
typedef struct out_wsm
{
    unsigned char       radio_number;
    unsigned char       channel_number;
    unsigned char       data_rate;
    signed char         txpwr_level;
    unsigned char       user_priority;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
    unsigned int        psid;
    unsigned long long  expiry_time;  /* Not supporting now */
    unsigned char       dest_mac[MAC_SIZE];
    struct {
        unsigned char   channel:1,
                        rate:1,
                        power:1,
                        load:1,
                        reserved:4;
    } extensions;
    unsigned char       data[WSM_MAX_SIZE];
    unsigned short      length;
} wme_out_wsm_t;
```

Struct Field Description:

Field	Description
radio_number	Valid value: RADIO_0 or RADIO_1
channel_ number	Valid value: 172, 174, 176, 178, 180, 182, 184.
data_rate	The data rate in the packet Valid value: 6, 9, 12, 18, 24, 36, 48, 54 (unit: Mb/s)
txpwr_level	The tx power in the sending packet Valid value: 0 ~ 25 unit:dBm
user_priority	The priority in the sending packet. The value maps to corresponding EDCA. Valid value: 0 ~ 7
psid	Provider Service Identifier (PSID) Valid value: defined as 1609.3
expiry_time	Not supporting now
dest_mac	The peer's MAC address

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

extensions	It indicates which of the WSMP header extension fields should be included in the packet.
data	The payload of the sending WSM packet
length	The length of the sending WSM packet

5.6.2. WSM Receiving

This function is used to receive WSM packets after registering the WSM service with the desired PSID. Before receiving, a valid handle ID should be obtained from "`wme_init`".

Prototype:

```
int wme_wsm_recv(wme_handle_t *handle, struct in_wsm *wsm,
unsigned int timeout);
```

Name	*wme_wsm_recv*		
Description	Receive WSM packet		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "***wme_init***"
wsm	Output	struct in_wsm *	WSM packet to receive
timeout	Input	unsigned int	Timeout of waiting receiving packets 0xFFFFFFFF implies wait forever

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Structure Type:

```
typedef struct in_wsm
{
    unsigned char       version;
    unsigned char       radio_number;
    unsigned char       channel_number;
    unsigned char       data_rate;
    signed char         txpwr_level;
     unsigned char        user_priority;  /* Not supporting now
*/
    unsigned int        psid;
    unsigned char       src_mac[MAC_SIZE];
    struct {
        unsigned char   channel:1,
                        rate:1,
                        power:1,
                        load:1,
                        reserved:4;
    } extensions;
    signed char         rssi;
    unsigned char       real_channel;
    unsigned char       data[WSM_MAX_SIZE];
    unsigned short      length;
}wme_in_wsm_t;
```

Struct Field Description:

Field	Description
version	The version of WSM
radio_number	Not supported currently.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

channel_ number	The channel number in the extension field of the received wsm packet Valid value: 172, 174, 176, 178, 180, 182, 184.
data_rate	The data rate in the extension field of the received WSM packet Valid value: 6, 9, 12, 18, 24, 36, 48, 54 (unit: Mb/s)
txpwr_level	The tx power in the extension field of the received WSM packet
user_priority	The priority in the received packet. The value maps to corresponding EDCA. Valid value: 0 ~ 7 Not supported currently.
psid	Provider Service Identifier (PSID) Valid value: defined as 1609.3
src_mac	The peer's MAC address
extensions	It indicates which of the WSMP header extension fields should be included in the packet.
rssi	The rssi of the received WSM packet (unit: dBm)
real_channel	The real channel number of the received WSM packet
data	The payload of the received WSM packet
length	The length of the received WSM packet

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.7. Event Receive API

This function is used to receive events (ex, available channels, or upcoming matching services) if corresponding services are registered. Before receiving, a valid handle ID should be obtained from "`wme_init`".

Prototype:

```
int wme_event_recv(wme_handle_t *handle, struct event_message *event, unsigned int timeout);
```

Name	*wme_event_recv*		
Description	Receive event message		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
handle	Input	wme_handle_t *	Handle ID return from "***wme_init***"
event	Output	struct event_message *	Event message to receive
timeout	Input	unsigned int	Timeout of waiting receiving events 0xFFFFFFFF implies wait forever

Structure Type:

```
typedef struct event_message
{
    unsigned char       event;
    unsigned char       reason;
    union {
        wme_event_channel_t      channel;
        wme_available_serv_mib_t service;
    } info;
} wme_event_message_t;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Struct Field Description:

Field	Description
event	Valid value: EVENT_SERVICE, EVENT_CHANNEL
reason	Valid value: REASON_SERVICE_AVAILABLE, REASON_SERVICE_UNAVAILABLE, REASON_CHANNEL_AVAILABLE, REASON_CHANNEL_UNAVAILABLE
info	The additional information of the event

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.8. MIB Information Get API

To get MIB information from WME

Prototype:

```
int wme_mib_get(struct mib_info *info);
```

Name	wme_event_recv		
Description	Receive event message		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
info	Output	struct mib_info *	MIB information get from WME

Structure Type:

```
typedef struct mib_info
{
    unsigned char       entry_type;
    unsigned char       next;  /* true: fetch the next available
index, false: fetch the index */
    short               entry_index;
    union {
        wme_provider_serv_mib_t    provider_entry;
        wme_user_serv_mib_t         user_entry;
        wme_channel_serv_mib_t      channel_entry;
        wme_wsm_serv_mib_t          wsm_entry;
        wme_available_serv_mib_t    available_entry;
    } entry_value;
} wme_mib_entry_t;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Struct Field Description:

Field	Description
entry_type	Valid value: PROVIDER_ENTRY, USER_ENTRY, CHANNEL_ENTRY, WSM_ENTRY, AVAILABLE_ENTRY
next	Valid value: 0, 1 0: Function will return MIB value by the entry_inedx 1: Function will search the next index after the entry_index
entry_index	The index of a specific mib entry
entry_value	The value of a specific mib entry

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

5.9. Example Code

Please refer to "`/example/test/cmd_dot3_v3.c`" for examples of APIs usage.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

6. IEEE 1609.2 Application Programming Interfaces (APIs)

This chapter describe the detail of APIs provided by security service. The process for using IEEE 1609.2 security service API is shown in Figure 2

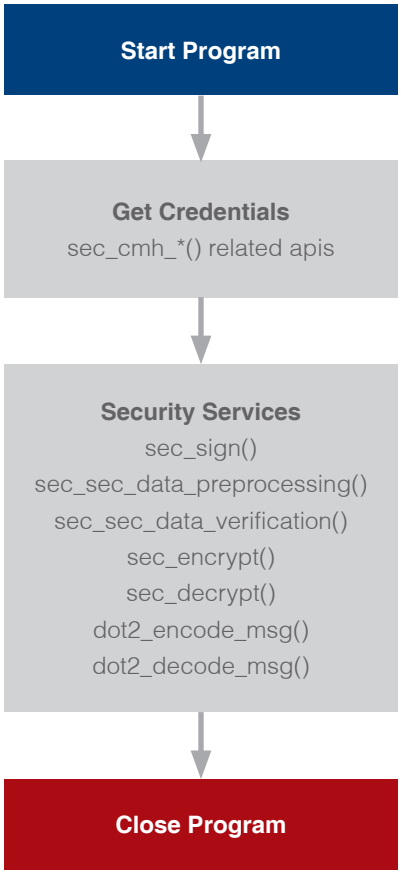


Figure 2 - Process of Security APIs

There are two parts of the APIs: (1) security credential related and (2) security service related.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

1. Security credential related APIs

For signing/verifying/encrypting/decrypting secure messages, the application would need corresponding certificates and corresponding keys. There are two ways to get the certificate and keys: load existing ones or generate/request new ones. The protocol stack supports APIs needed for implementing both by user for getting the security credentials. The details of these APIs are described in Section 6.1

2. Security service related APIs

These APIs are used to encode/decode secure messages and the details are described in Section 6.2

The function prototype and usage data structures, enumerate values are defined in the header file "`/include/dot2_api.h`".

6.1. Security Credential Management

6.1.1. Cryptomaterial Management

The protocol stack provides the APIs to manage security credentials according to the IEEE 1609.2 standard Section 9.2.2. User can use these APIs to implement the desired scheme based on their applications. Figure 3 describes the transitions between each state

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

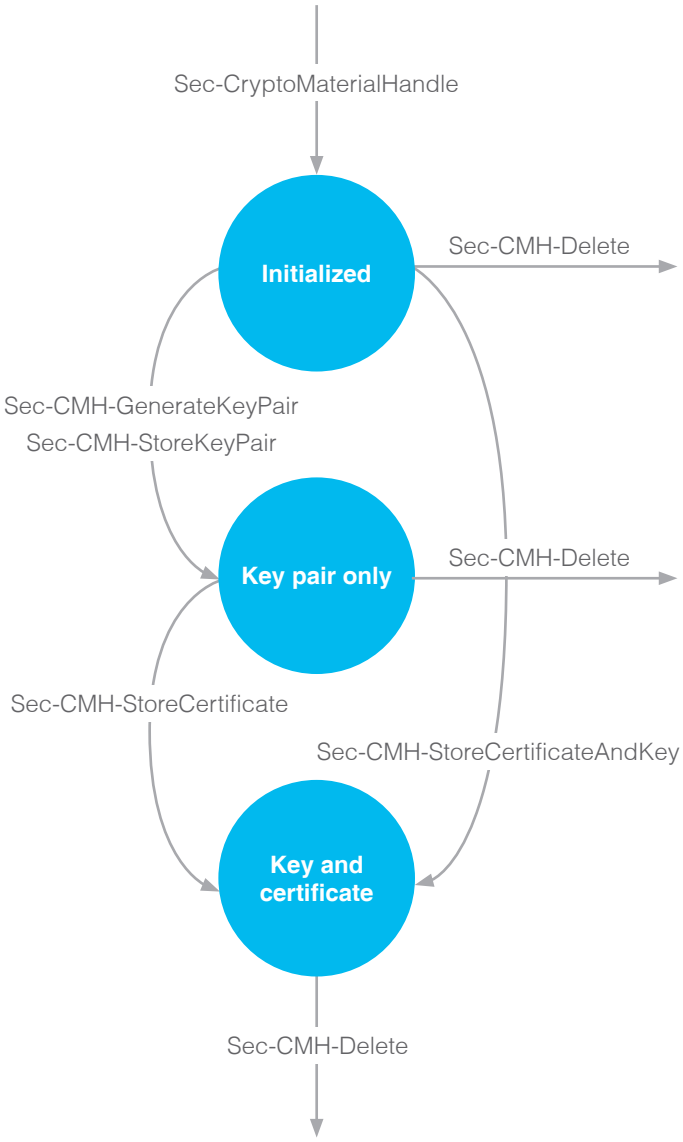


Figure 3 - Credential State Transit

Following are relative APIs according to the credential state transit

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Cryptomaterial Creation:

int sec_cmh_new(unsigned int o_cmh);*

Name	*sec_cmh_new*		
Description	This primitive creates a new cryptomaterial handle and set into the initialized state.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
o_cmh	Output	unsigned int*	The cryptomaterial handle created

Key Pair Generation:

int sec_cmh_gen_key_pair(unsigned int i_cmh, dot2_ec_algorithms_t i_alg, private_key_type_t i_key_type, unsigned char o_pub_key);*

Name	*sec_cmh_gen_key_pair*		
Description	The primitive would generate a key pair according to the input algorithm, key type and transform the referenced CMH to key pair only state. NOTE: If the key type was set to RECONSTRUCT_SEED, it may need to call sec_cmh_store_cert() primitive to reconstruct the actual private key. This is used to support acquiring certificate and keys from a PKI such as SCMS scheme.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_cmh	Input	unsigned int*	The cryptomaterial handle that would be bound with the generated key pair
i_cert_len	Input	dot2_ec_algorithms_t	The elliptic curve and algorithm used for generating the key pair
i_key_type	Input	private_key_type_t	The usage of the private key
o_pub_key	Output	unsigned char*	The generated public key

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Key Pair Store:

int sec_cmh_store_key_pair(unsigned int i_cmh, dot2_ec_algorithms_t i_alg, unsigned char i_pub_key, unsigned char* i_pri_key);*

Name	*sec_cmh_gen_key_pair*		
Description	The primitive would store the input key pair according to the input algorithm and transform the referenced CMH to key pair only state.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_cmh	Input	unsigned int*	The cryptomaterial handle that would be bound with the generated key pair
i_alg	Input	dot2_ec_algorithms_t	The elliptic curve and algorithm used for the imported key pair
i_pub_key	Input	unsigned char*	The public key to be stored
i_pri_key	Input	unsigned char*	The private key to be stored

Certificate Store:

int sec_cmh_store_cert(unsigned int i_cmh, unsigned int i_cert_len, unsigned char i_cert, unsigned char* i_pri_key_material);*

Name	*sec_cmh_store_cert*
Description	The primitive would store and bind the input certificate with the input CMH. The parameter i_pri_key_material would be ignored if the certificate is explicit type and the corresponding private key would be the one previously generated or stored by calling sec_cmh_gen_key_pair() or sec_cmh_store_key_pair(). If the certificate is implicit type, the security module would try to reconstruct the private key using the i_pri_key_material and the seed generated previously by calling sec_cmh_gen_key_pair().

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_cmh	Input	unsigned int*	The cryptomaterial handle that would be bound with the generated key pair
i_cert_len	Input	unsigned int	The length in bytes of the certificate to be imported
i_cert	Input	unsigned char*	The string of the certificate to be imported
i_pri_key_ material	Input	unsigned char*	The private reconstruction value to be used to reconstruct the actual private key

Certificate and Key Store:

int sec_cmh_store_cert_and_key(unsigned int i_cmh, unsigned int i_cert_len, unsigned char i_cert, unsigned char* i_pri_key);*

Name	*sec_cmh_store_cert_and_key*		
Description	The primitive would store the certificate and key and transform the referenced CMH to key and certificate state.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_cmh	Input	unsigned int*	The cryptomaterial handle that would be bound with the generated key pair
i_cert_len	Input	unsigned int	The length in bytes of the certificate to be imported
i_cert	Input	unsigned char*	The string of the certificate to be imported
i_pri_key	Input	unsigned char*	The private key to be stored

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Cryptomaterial Deletion:

int sec_cmh_del(unsigned int i_cmh);

Name	*sec_cmh_del*		
Description	The primitive would delete the referenced cryptomaterial handle and related data.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_cmh	Output	unsigned int*	The cryptomaterial handle that would be deleted

6.1.2. Symmetric Cryptomaterial Management

The protocol stack provides the APIs to manage symmetric cryptomaterial handles according to the IEEE 1609.2 standard section 9.2.3.

Symmetric Cryptomaterial Creation:

int sec_scmh_new(dot2_sym_algorithms_t i_alg, unsigned char i_gen_flag, unsigned char i_sym_key, unsigned int* o_scmh);*

Name	*sec_scmh_new*		
Description	This primitive creates a newly symmetric cryptomaterial handle and sets the related symmetric key.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_alg	Input	dot2_sym_algorithms_t	The symmetric algorithm used with the symmetric key
i_gen_flag	Input	unsigned char	The flag to generate a symmetric key when allocate a new SCMH or not

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

i_sym_key	Input	unsigned char	If i_gen_flag == 0, then importing the i_sym_key as the symmetric key
o_scmh	Output	unsigned int*	The created symmetric cryptomaterial handle

Symmetric Cryptomaterial Hash:

int sec_scmh_hashid8(unsigned int i_scmh, unsigned char o_hashid8);*

Name	*sec_scmh_hashid8*		
Description	The primitive would hash the symmetric key bound with the input SCMH and set the value to the output parameter o_hashid8.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_scmh	Input	unsigned int	The symmetric cryptomaterial handle to get the related hash value of the corresponding symmetric key
o_hashid8	Output	unsigned char*	The least significant 8 bytes of the hash value of the symmetric key

Symmetric Cryptomaterial Deletion:

int sec_scmh_del(unsigned int i_scmh);

Name	*sec_scmh_del*		
Description	The primitive would delete the referenced symmetric cryptomaterial handle and related data.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_scmh	Input	unsigned int	The symmetric cryptomaterial handle that would be deleted

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

6.1.3. CA Certificate

The protocol stack also provides an API for importing CA certificates. The added certificates would be recognized as trust anchors when verifying the secure messages.

Certificate Store:

int sec_cmh_store_ca_cert(unsigned int i_cert_len, unsigned char i_cert, unsigned int* o_cmh);*

Name	*sec_cmh_store_ca_cert*		
Description	The primitive would import CA certificates. The added certificates would be recognized as trust anchors when verifying the secure messages.		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_scmh	Input	unsigned int	The symmetric cryptomaterial handle that would be deleted
i_cert	Input	unsigned char*	The content of the certificate
o_cmh	Output	unsigned int*	The cryptomaterial handle created

6.1.4. Certificate Generation

To ease the testing, the stack provides another API to generate a certificate and related key with most fields set as default values

Certificate Generation:

int dot2_generate_cert(unsigned int i_issuer_cmh, dot2_cert_type_t i_cert_type, unsigned int i_psid, unsigned int

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

**o_cmh, unsigned int* o_cert_len, unsigned char* o_cert_str);*

Name	*dot2_generate_cert*		
Description	Generate a certificate and related key with most fields set as default values		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_issuer_cmh	Input	unsigned int	The cryptomaterial handle of the issue certificate
i_cert_type	Input	dot2_cert_type_t	Certificate type to be generated
i_psid	Input	unsigned int	The permitted PSID of the generated certificat
o_cmh	Output	unsigned int *	The cryptomaterial handle created
o_cert_len	Output	unsigned int *	The length of the certificate
o_cert_str	Output	unsigned char*	The content of the certificate

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

6.2. Security Message Service

6.2.1. Signed Data Generation

Prototype:

```
int sec_sign(sec_sign_t* sign_info);
```

Name	sec_sign		
Description	To generate 1609.2 signed data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
sign_info	Input	sec_sign_t*	Parameters of sign

Structure Type:

```
typedef struct {
  unsigned int cmh;
  unsigned int data_len;
  unsigned char* data;
  dot2_data_type_t data_type;                    //      RAW      or
Ieee1609Dot2Data
  unsigned char ext_data_hash[32];
  unsigned char ext_data_hash_alg;          // should be sha256
  unsigned int psid;
  unsigned char set_gen_time;                    // true or false
  unsigned char set_gen_loc;                     // true or false
  unsigned int exp_time;                  // time duration in ms
  dot2_signer_type_t signer_id_type;          //   cert,   digest,
self
  char signer_id_cert_chain_len;       // 1~256, 0 for MAX
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
  unsigned char max_cert_chain_len;         //     should     be
positivie integer >= 2
  dot2_ec_point_choice_t sign_with_fast_ver;     //         y/w
uncompressed, y/w compressed, no
  dot2_ec_point_choice_t ecpoint_format;    //     uncompressed,
compressed
  unsigned char use_p2p_cert_dist;          // true or false
  unsigned int sdee_id;

  unsigned int signed_data_len;             // in: max length of
signed_data;
                                  // out: actual length of signed_
data;
  unsigned char* signed_data;
} sec_sign_t;
```

Struct Field Description:

Field	Description
cmh	The primitive would use the certificate and key bound with input CMH to sign the message.
data_len	The length in bytes of the input data.
data	The data to be signed.
data_type	The type should be UNSECURE or SIGNED. If the data type is UNSECURE, then the primitive would pack the input data into an Ieee1609Dot2Data structure before encoding it into a signed message.
ext_data_hash	Reserved, currently not used.
ext_data_ hash_alg	Reserved, currently not used.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

psid	The PSID of the application which uses the 1609.2 secure messages. The encoding of PSID would follow the encoding rules defined in IEEE 1609.3.
set_gen_time	The value of the parameter should be 0 or 1. It indicates the primitive would encode the generation time field or not.
set_gen_loc	The value of the parameter should be 0 or 1. It indicates the primitive would encode the generation location field or not.
exp_time	If the value is not zero, then the primitive would encode the expiration time field in the header and the expiration time would be set to the current time + the amount of exp_time (unit: ms).
signer_id_type	The value of the parameter should be DIGEST or CERTIFICATE. It indicates the signer identifier type of the signed message.
signer_id_ cert_chain_len	Reserved, currently only encode the signer certificate.
max_cert_ chain_len	Reserved, the value of the max_cert_chain_len should be larger than signer_id_cert_chain_len.
sign_with_ fast_ver	The value of the parameter should be one of X_ONLY, COMPRESSED, and UN COMPRESSED. It indicates the primitive would encode the R value of the signature field with x-coordinate only, compressed point or uncompressed point type.
ecpoint_format	Reserved, currently not used.
use_p2p_cert_ dist	Reserved, currently not used.
sdee_id	Reserved, currently not used.
signed_data_ len	The length in bytes of the encoded signed message. NOTE: when calling the primitive, the signed_data_len should be set to the value of the maximum length of signed_data. If the encoded length is larger than the input value, it would fail to encode the signed message. If the encoding process succeeded, the signed_data_len would be set to the actual encoded length of the signed data.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

signed_data	The encoded signed message. NOTE: the user should allocate enough space for the security module to fill the signed message before calling the primitive.

6.2.2. Data Preprocessing

Prototype:

```
int sec_sec_data_preprocessing(sec_sec_data_preprocessing_t* sec_data);
```

Name	*sec_sec_data_preprocessing*		
Description	Preprocess 1609.2 data before verification		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
sec_data	Input	sec_sec_data_preprocessing_t*	Parameters of preprocessing

Structure Type:

```
typedef struct {
  unsigned int data_len;
  unsigned char* data;
  unsigned int sdee_id;
  unsigned char use_p2p_cd;

  dot2_data_type_t content_type;
  unsigned int psid;
  unsigned int ssp_len;
  unsigned char ssp[32];
  unsigned char assurance_level;
  unsigned int next_crl_time;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
  unsigned int raw_data_len;            // in: max array
size; out: actual data size
  unsigned char* raw_data;
  sec_sec_data_verification_t data_info;     //        put
extracted info here
} sec_sec_data_preprocessing_t;
```

Struct Field Description:

Field	Description
data_len	The length in bytes of the input data.
data	The data to be decoded.
sdee_id	Reserved, currently not used.
use_p2p_cert_dist	Reserved, currently not used.
content_type	The extracted data type of the input data.
psid	The extracted PSID of the input data from the header field.
ssp_len	The corresponding ssp information related to the permitted PSID of the signer certificate.
ssp	The corresponding ssp information related to the permitted PSID of the signer certificate.
assurance_level	Reserved, currently not used.
next_crl_time	Reserved, currently not used.
raw_data_len	The length in bytes of the decoded data. NOTE: when calling the primitive, the raw_data_len should be set to the value of the maximum length of raw_data. If the extracted data length is larger than the input value, it would fail to decode the secure message. If the decoding process succeeded, the raw_data_len would be set to the actual decoded length of the secure data.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

raw_data	The extracted data. NOTE: the user should allocate enough space for the security module to fill the extracted message before calling the primitive.
data_info	The structure may be used later for verification. The primitive would fill the extracted values from the header fields of the secure message.

6.2.3. Verification

Prototype:

```
int sec_sec_data_verification(sec_sec_data_verification_t*
sec_data);
```

Name	*sec_sec_data_verification*		
Description	Verify 1609.2 data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
sec_data	Input	sec_sec_data_verification_t*	Parameters of verification

Structure Type:

```
typedef struct {
  unsigned int sdee_id;
  unsigned int psid;
  dot2_data_type_t content_type;
  unsigned int signed_data_len;
  unsigned char* signed_data;
  unsigned char external_data_hash[32];
  unsigned char hash_alg;
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
   unsigned char max_cert_chain_len;        // should be
positivie integer >= 2
  unsigned char data_hash_for_replay_check[32];
  unsigned char replay_check;      // true or false
  unsigned long long gen_time;          //   provided   if
header not contain gentime
  unsigned char gen_time_in_past_check; // if true, reject
too-old SPDU
  unsigned int validity_period;         //   provided   if
gentimeinpast = true
  unsigned char gen_time_in_future_check;    //  if  true,
reject future SPDU
  unsigned int future_data_period; //     provided     if
gentimeinfuture = true
  unsigned char expiry_check;      //   if   true,   reject
expired SPDU
  unsigned long long expiry_time;  //     provided     if
expirycheck = true
  unsigned char gen_loc_in_cert_check;  // check genloc is
inside the region of cert
  unsigned char gen_loc_dis_check; //   check   genloc   is
inside the relevance range
  double validity_dis;             //    validity    range
between recp and genloc (m)
  _3dloc gen_loc;             // provided if header not
contain genloc
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
  unsigned int crl_tolerance;
  unsigned char cert_exp_check;          // if true, reject
if any one of cert in chain is expired

  unsigned char signerid8[8];
  signature_t sig;
} sec_sec_data_verification_t;
```

Struct Field Description:

Field	Description
sdee_id	Reserved, currently not used.
psid	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the header field has the value, then the value would be set by the primitive. The PSID value would be used to check whether the signer has the permission or not.
content_type	Reserved, currently not used.
signed_data_len	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the secure message is a signed message, then the value would be set by the primitive. The signed data would be used to verify the signature.
signed_data	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the secure message is a signed message, then the content would be set by the primitive. The signed data would be used to verify the signature.
external_data_hash	Reserved, currently not used.
hash_alg	Reserved, currently not used.
max_cert_chain_len	Reserved, currently not used.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

data_hash_for_ replay_check	If the structure is one of the input parameter of sec_sec_ data_preprocessing() called before and the secure message is a signed message, then the content would be set by the primitive. If the replay_check is set to 1, the content would be used to check if it is a replay of previous message.
replay_check	The value should be 0 or 1. It indicates whether to perform the replay check or not.
gen_time	If the structure is one of the input parameter of sec_sec_ data_preprocessing() called before and the header field has the value, then the value would be set by the primitive. The value would be used to perform generation time related checks.
gen_time_in_past_ check	The value should be 0 or 1. It indicates whether to check the generation time is in past or not.
validity_period	The range of the allowable period for checking the generation time is too old or not. The unit is in microsecond.
gen_time_in_future_ check	The value should be 0 or 1. It indicates whether to check the generation time is in future or not.
future_data_period	The range of the allowable period for checking the generation time is in the future or not. The unit is in microsecond.
expiry_check	The value should be 0 or 1. It indicates whether to check the expiration time or not.
expiry_time	If the structure is one of the input parameter of sec_sec_ data_preprocessing() called before and the header field has the value, then the value would be set by the primitive. The value would be used to perform expiry check.
gen_loc_in_cert_ check	The value should be 0 or 1. It indicates whether to check if the generation location is in the permitted range of the signer or not.
gen_loc_dis_check	The value should be 0 or 1. It indicates whether to check if the generation location is too far or not.
validity_dis	The allowable distance for checking the generation location is nearby or not. The unit is in meter.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

gen_loc	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the header field has the value, then the value would be set by the primitive. The value would be used to perform generation location related checks.
crl_tolerance	Reserved, currently not used.
cert_exp_check	The value should be 0 or 1. It indicates whether to check the expiration time of the signer certificate chain or not.
signerid8	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the header field has theinformation of the signer, then the value would be set by the primitive. The signerid8 would be used to indicate the signer and check related fields of the signer certificate.
sig	If the structure is one of the input parameter of sec_sec_data_preprocessing() called before and the secure message is a signed message, then the signature would be set by the primitive. The signature would be used to check the validity of the message.

6.2.4. Encryption

Prototype:

```
int sec_encrypt(sec_encrypt_t* encrypt_info);
```

Name	*sec_encrypt*		
Description	To generate 1609.2 encrypted data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
encrypt_info	Input	sec_encrypt_t*	Parameters of encryption

Structure Type:

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

```
typedef struct {
  unsigned int data_len;
  unsigned char* data;
  dot2_data_type_t data_type;      //        RAW        or
Ieee1609Dot2Data
  unsigned char enc_key_type;      // static or ephemeral
  unsigned int scmh;
  unsigned int recp_cert_num;
  unsigned char* recp_cert_id_list;     //  list  of  recp
cert id (hashid8)
  unsigned char* signed_data_recp_info;
  unsigned char resp_enc_key[8];
  dot2_ec_point_choice_t ecpoint_format;

  unsigned int enc_data_len;       //  in:  max  length  of
enc_data;
                          //  out:  actual  length  of  enc_
data;
  unsigned char* enc_data;
  unsigned int fail_cert_num;
  unsigned char* fail_cert_id_list;     // list of failed
cert id (hashid8)
} sec_encrypt_t;
```

Struct Field Description:

Field	Description
data_len	The length in bytes of the input data.
data	The data to be encrypted.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

data_type	If the data type is UNSECURE, then the primitive would pack the input data into an Ieee1609Dot2Data structure before encoding it into an encrypted message.
enc_key_type	Reserved, currently not used.
scmh	Reserved, currently not used.
recp_cert_num	The number of recipients.
recp_cert_id_list	The array of certificate id (hashid8) of the recipients.
signed_data_recp_info	Reserved, currently not used.
resp_enc_key	Reserved, currently not used.
ecpoint_format	The value should be COMPRESSED or UNCOMPRESSED. It indicates the point format for encoding the RecipientInfos.
enc_data_len	The length in bytes of the encrypted data. NOTE: when calling the primitive, the enc_data_len should be set to the value of the maximum length of enc_data. If the encrypted data length is larger than the input value, it would fail to encrypt the message. If the encryption succeeded, the enc_data_len would be set to the actual length of the encrypted data.
enc_data	The encrypted data. NOTE: the user should allocate enough space for the security module to fill the encrypted message before calling the primitive.
fail_cert_num	The number of recipients with which failed to encrypt.
fail_cert_id_list	The array of certificate id (hashid8) of the recipients with which failed to encrypt.

6.2.5. Decryption

Prototype:

```
int sec_decrypt(sec_decrypt_t* decrypt_info);
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Name	*sec_decrypt*		
Description	To decrpyt 1609.2 data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
decrypt_info	Input	sec_decrypt_t*	Parameters of decryption

Structure Type:

```
typedef struct {
  unsigned int data_len;
  unsigned char* data;
  unsigned int cmh;
  unsigned char signed_data_recp_info[32];

  unsigned int decrypted_data_len; // in: max length of decrypted_data;
                          // out: actual length of decrypted_data;
  unsigned char* decrypted_data;
} sec_decrypt_t;
```

Struct Field Description:

Field	Description

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

data_len	The length in bytes of the input data.
data	The data to be encrypted.
cmh	The primitive would use the certificate and key bound with input CMH to decrypt the message.
signed_data_recp_info	Reserved, currently not used.
decrypted_data_len	The length in bytes of the decrypted data. NOTE: when calling the primitive, the decrypted_data_len should be set to the value of the maximum length of decrypted_data. If the decrypted data length is larger than the input value, it would fail to decrypt the message. If the decryption succeeded, the decrypted_data_len would be set to the actual length of the decrypted data.
decrypted_data	The decrypted data. NOTE: the user should allocate enough space for the security module to fill the decrypted message before calling the primitive.

6.2.6. Encoding

The protocol stack also provides a primitive to encode secure messages more easily if user does not want to setup lots of parameters.

Prototype:

```
int dot2_encode_msg(unsigned int i_psid, dot2_encode_msg_t* para);
```

Name	*dot2_encode_msg*		
Description	To encode 1609.2 data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_psid	Input	unsigned int i	The PSID of encoded data
para	Input	dot2_encode_msg_t*	Parameters of encoding

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Structure Type:

```
typedef struct {
  unsigned int i_cmh;
  unsigned char i_sign_flag;
  unsigned char i_enc_flag;
  unsigned int i_enc_recp_num;
  unsigned int i_enc_recp_cmh[8];
  unsigned int i_msg_len;
  unsigned char* i_msg;
  unsigned int i_lifetime;    // units: ms
  unsigned int o_msg_len;
  unsigned char* o_msg;
  unsigned int fail_cert_num;
  unsigned char* fail_cert_id_list;
} dot2_encode_msg_t;
```

Struct Field Description:

Field	Description
i_cmh	The primitive would use the certificate and key bound with input CMH to encode the message.
i_sign_flag	The value indicates whether to sign the message or not.
i_enc_flag	The value indicates whether to encrypt the message or not.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

i_enc_recp_num	The number of recipients, used if enc_flag is set
i_enc_recp_cmh	The array of certificate id (hashid8) of the recipients, used if enc_flag is set
i_msg_len	The length in bytes of the input data.
i_msg	The data to be encoded.
i_lifetime	If the input parameter psid is not BSM_PSID, then the primitive would use the value of the i_lifetime as the expiration field in the header and the expiration time would be set to the current time + the amount of i_lifetime (unit: ms).
o_msg_len	The length in bytes of the encoded data. NOTE: when calling the primitive, the o_msg_len should be set to the value of the maximum length of o_msg. If the encoded data length is larger than the input value, it would fail to encode the message. If the encoding process succeeded, the o_msg_len would be set to the actual length of the encoded data.
o_msg	The encoded data. NOTE: the user should allocate enough space for the security module to fill the encoded message before calling the primitive.

6.2.7. Decoding

The protocol stack also provides a primitive to decode secure messages more easily if user does not want to setup lots of parameters.

Prototype:

```
int dot2_decode_msg(unsigned int i_psid, dot2_decode_msg_t* para);
```

Name	dot2_decode_msg

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Description	To decode 1609.2 data		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
i_psid	Input	unsigned int i	The PSID of decode data
para	Input	dot2_decode_msg_t*	Parameters of decoding

Structure Type:

```
typedef struct {
  unsigned char i_verify_flag;
  unsigned int i_msg_len;
  unsigned char* i_msg;
  unsigned int o_msg_len;
  unsigned char* o_msg;
  unsigned long long o_generation_time;
  unsigned long long o_expiration_time;
  unsigned int o_next_crl_time;
} dot2_decode_msg_t;
```

Struct Field Description:

Field	Description
i_verify_flag	The value indicates whether to verify the message or not if it is a signed message.
i_msg_len	The length in bytes of the input data.
i_msg	The data to be decoded.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

o_msg_len	The length in bytes of the decoded data. NOTE: when calling the primitive, the o_msg_len should be set to the value of the maximum length of o_msg. If the decoded data length is larger than the input value, it would fail to decode the message. If the decoding process succeeded, the o_msg_len would be set to the actual length of the decoded data.
o_msg	The decoded data. NOTE: the user should allocate enough space for the security module to fill the decoded message before calling the primitive.
o_generation_time	If decoded successfully, the o_generation_time would be set to the value retrieved from the secure header if the header has that field.
o_expiration_time	If decoded successfully, the o_expiration_time would be set to the value retrieved from the secure header if the header has that field.
o_next_crl_time	Reserved, currently not used.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

6.3. Example Code

Please refer to "`/example/test/cmd_test_us_sec.c`" for examples of APIs usage.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

7. SAE J2735 Application Programming Interfaces (APIs)

To support interoperability among DSRC applications, SAE J2735 standard defines standardized messages. In this document we demonstrate one of SAE J2735 message, the Basic Safety Message (BSM), as per release version 63.

7.1. Basic Safety Message (BSM)

The basic safety message (BSM) encoding, using ASN.1 PER encoding, is defined below. Part I data shall be included in every BSM. Part II data items are optional for a given BSM and are included as needed according to policies that are beyond the scope of J2735 standard. A BSM without Part II optional content is a valid message.

```
BasicSafetyMessage ::= SEQUENCE {
   -- Part I, Sent at all times with each message
   coreData      BSMcoreData,
   -- Part II Content
   partII  SEQUENCE (SIZE(1..8)) OF
           PartIIcontent {{ BSMpartIIExtension }} OPTIONAL,
   regional SEQUENCE (SIZE(1..4)) OF
           RegionalExtension  {{REGION.Reg-BasicSafetyMessage}}
           OPTIONAL,
...
}
```

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

7.1.1. BSM Encoding

Prototype:

```
int  j2735r63_bsm_encode(bsm_standard_item  *encode_item,
unsigned char encode_buf[], size_t input_buf_size, size_t
*output_buf_size, unsigned char print_level);
```

Name	*j2735r63_bsm_encode*		
Description	To encode basic safety message		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
encode_item	Input	bsm_standard_item *	The input BSM data elements.
encode_buf	Output	unsigned char *	If the return value is 0, encode_buf will be a BSM encoded buffer.
input_buf_size	Input	size_t	input_buf_size is the size of the encode_buf.
output_buf_size	Output	size_t *	If the return value is 0, output_buf_size will be the size of the BSM encoded buffer.
print_level	Input	unsigned char	PRINT_NON: print nothing PRINT_BASIC: print encoded BSM struct

For structure type "`bsm_standard_item`" detail, please refer to "`/include/j2735r63_bsm_api.h`" and relative header files. Also refer to SAE-J2735 standard.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

7.1.2. BSM Decoding

Prototype:

```
int j2735r63_bsm_decode( bsm_standard_item **input_decoded_item, unsigned char decode_buf[], size_t decode_buf_size, unsigned char print_level);
```

Name	*j2735r63_bsm_decode*		
Description	To decode basic safety message		
Return Value	0: Success Other: Fail		
Parameters	**Mode**	**Type**	**Parameter description**
input_decoded_item	Output	bsm_standard_item **	If the return value is 0, input_decoded_item will be the decoded bsm_standard_item.
decode_buf	Input	unsigned char *	The BSM encoded buffer
decode_buf_size	Input	size_t	The size of the decode_buf[]
print_level	Input	unsigned char	PRINT_NON: print nothing PRINT_BASIC: print decoded BSM struct

7.1.3. Example Code

Please refer to "`/example/test/cmd_j2735r63_bsm.c`" for examples of APIs usage.

A printed version of this document is an uncontrolled copy
© 2018 Unex Technology Corporation - Company Confidential

Unex Technology Corporation

11F-3, No. 100, Sec. 1, Jiafeng 11th Rd., Zhubei City,

Hsinchu County 302, Taiwan, R.O.C.

Tel. 886-3-657-8188

Fax. 886-3-657-8199

www.unex.com.tw